

NEXUS AI

Application / UI Design Document

N46–N54 • UX/UI • Frontend • Desktop • Workspace • Chat • Models • Knowledge • Agents • Settings

Version: 0.1 **Status:** Product Interface Foundation **Design Goal:** Powerful but approachable local AI workstation

Purpose: Define the user-facing application structure of NEXUS AI. The interface must make advanced local AI, RAG, model routing and agent execution understandable without hiding important controls from power users.

APP SHELL → WORKSPACE → CHAT / KNOWLEDGE / MODELS / AGENTS → SETTINGS & PRIVACY

1. Application Design Vision

NEXUS AI should feel like a modern AI workstation rather than a collection of disconnected utilities. Chat is the primary entry point, while projects, knowledge, models and agents are integrated into the same workspace.

Design Principles

- Local-first visibility: clearly communicate when processing is local/offline.
- Progressive complexity: simple chat first, advanced controls when needed.
- Workspace continuity: project context follows the user across chat, files, knowledge and agents.
- Observable AI: show model, sources, tool activity and task status without exposing hidden chain-of-thought.
- Fast interaction: streaming responses, responsive navigation and non-blocking long tasks.
- User control: permissions, privacy and model selection remain accessible.
- Professional technical aesthetic: focused, clean, high information density without visual clutter.

2. N46 — UX/UI Design

The core application shell uses a three-region structure: navigation, main workspace and contextual panel.

Region	Purpose
Left Navigation	Home, Chat, Projects, Knowledge, Models, Agents, Settings.
Main Workspace	Current page, conversation or task.
Right Context Panel	Sources, task details, model information, tool activity or project context when relevant.
Global Header	Current project, local runtime status, search and user/application controls.
Composer / Action Bar	Chat input, model selector, attachments, tools and execution controls.

Visual Hierarchy

Primary action → task content → supporting context → advanced diagnostics. Advanced information should be collapsible instead of removed.

States

Every major screen must define loading, empty, success, error, offline, permission-required and unavailable states.

3. Information Architecture

Primary Area	Subsections
Home	Recent projects, recent conversations, model status, quick actions.
Chat	All chats, current conversation, search, pinned conversations.
Projects	Project list, workspace, files, knowledge, tasks.
Knowledge	Collections, documents, ingestion jobs, search.
Models	Installed models, registry, benchmark, health, routing.
Agents	Tasks, active runs, history, agent roles, tool activity.
Settings	General, models, storage, permissions, privacy, appearance, advanced.

4. N47 — Frontend Architecture

The frontend is organized around reusable application shells, feature modules and shared UI primitives.

Frontend Layer	Responsibility
App Shell	Navigation, routing, global status and layout.
Feature Modules	Chat, Projects, Knowledge, Models, Agents, Settings.

Frontend Layer	Responsibility
State Layer	Session, project, chat, task and UI state.
API Layer	Typed requests, streaming events, errors and retries.
UI System	Buttons, cards, dialogs, tables, tabs, panels, editors.
Utilities	Formatting, validation, permissions display and local helpers.

Frontend State Separation

Server/application state: conversations, models, projects, documents, tasks. Local UI state: panel visibility, selected tab, composer state, filters. Long-running task state arrives through events and is persisted by the backend.

5. N48 — Desktop Application Architecture

NEXUS is intended to be installable on the user's device and operate as a local application.

Desktop Shell → Frontend UI → Local Application API → Orchestrator → Ollama / RAG / Tools / Database / Workspace.

Desktop Requirement	Design
Installation	Single-user local installation with application data directory.
Startup	Check runtime, database, storage and Ollama availability.
Offline	Core UI and local AI remain functional without internet.
Updates	Versioned application updates with safe recovery.
System tray/background	Optional for long-running agent tasks.
File associations	Optional future integration for supported knowledge files.
Data location	Clearly shown and configurable where practical.

Startup Experience

Launch → Runtime health check → Ollama/model discovery → database check → workspace check → ready state. Errors should provide actionable repair steps.

6. N49 — Workspace & Project System Design

Projects are the main unit of continuity. A project combines conversations, workspace files, knowledge collections, agent tasks and project memory.

Project Area	Contents
Overview	Description, activity, tasks and quick actions.
Chat	Project-scoped conversations.
Files	Controlled project filesystem.
Knowledge	Project knowledge collection(s).
Agents	Project tasks and execution history.
Memory	Approved project decisions/facts.
Settings	Project paths, permissions and preferences.

Project Creation

New Project → Name → Workspace location → Optional knowledge sources → Default model policy → Permissions → Create.

Project Isolation

A project must have an explicit filesystem root and knowledge scope. Cross-project access is disabled unless explicitly allowed.

7. N50 — Chat & Conversation System Design

Chat is the primary NEXUS interaction surface. It should support both simple model conversations and advanced orchestrated tasks without forcing users to understand the underlying architecture.

Chat Element	Behavior
Message Stream	Real-time streamed model response.
Model Indicator	Shows selected/automatically routed model.
Source Indicator	Shows when RAG evidence was used.
Tool Activity	Shows active/completed tool operations.
Task Status	Shows planning, execution, verification or waiting.
Composer	Text, attachments, project context and action controls.
Stop	Cancel active generation/task.
Retry	Retry with same or alternative model when allowed.
Branch/Continue	Optional future conversation branching.
Copy/Export	Copy response or export conversation.

Simple vs Agentic Chat

Simple question → direct response. Complex request → NEXUS can display a compact “working” state with stages such as Planning → Searching → Editing → Testing → Verifying. The UI should not expose hidden chain-of-thought.

Conversation Metadata

Store conversation ID, project, model, routing mode, sources, task references and timestamps. The visible UI should emphasize useful context rather than raw telemetry.

8. N51 — Model Control Center Design

The Model Center is the operational dashboard for local AI models.

Screen	Purpose
Installed Models	List Ollama models, size, status and availability.
Model Details	Capabilities, context, resource profile and benchmark scores.
Routing	View/adjust role mappings and routing preferences.
Benchmark	Run benchmark suites and compare models.
Runtime	Loading status, active model, latency and resource signals.
Fallback	Configure eligible fallback models.

Model Card

Model name • provider • tag • size • capabilities • context • health • performance • preferred tasks • routing status.

Manual Override

Users can choose a model for a conversation/task. The UI must clearly indicate when automatic routing is disabled or overridden.

9. N52 — Knowledge Center UI Design

The Knowledge Center manages local documents and collections.

UI	Function
Collections	Create/manage knowledge boundaries.
Document List	Filename, version, status, size, collection and indexing state.
Import	Drag/drop or file picker for supported documents.
Ingestion Jobs	Progress, success/failure and diagnostics.
Search	Semantic/hybrid search with filters.

UI	Function
Document Viewer	Preview source content and metadata.
Evidence	Show retrieved chunks and source references.
Versions	Compare active/superseded versions.

Knowledge Status

Queued → Processing → Indexed → Active. Failure states should identify parser, embedding or index errors without exposing unnecessary technical complexity.

10. N53 — Agent Workspace UI Design

The Agent Workspace is the control room for long-running AI tasks.

Panel	Purpose
Task Header	Goal, project, agent role, model and current status.
Plan	High-level steps and progress.
Activity	Tool calls, retrieval, model operations and outcomes.
Files	Changed files and diffs.
Terminal	Command/test status and sanitized output.
Verification	Tests, evidence and completion checks.
Controls	Stop, pause where supported, approve, retry, resume.

Agent Status

Queued • Planning • Running • Waiting for approval • Waiting for tool • Testing • Verifying • Completed • Failed • Cancelled.

Approval UX

High-risk actions appear as explicit approval cards describing the operation, scope and expected effect. The user can Approve, Reject or Cancel the task.

11. N54 — Settings & Privacy UI Design

Settings must make the local-first architecture understandable and controllable.

Settings Area	Controls
General	Language, startup behavior, defaults.
Appearance	Theme, density, interface preferences.
Models	Ollama connection, model discovery, defaults, routing.
Storage	Database/workspace/knowledge locations and retention.
Permissions	Filesystem, terminal, network and integration policies.
Privacy	Offline mode, telemetry, logs, data handling.
Security	Sandbox policies, approvals, sensitive data handling.
Advanced	Diagnostics, export/import, reset and developer options.

Privacy Dashboard

Show clearly: Local processing status, internet connectivity policy, stored data locations, enabled integrations, telemetry status and active permissions.

Reset / Delete

Destructive settings require confirmation and clearly state what will be deleted. Project deletion and knowledge deletion should identify derived indexes and memory affected.

12. Design System

The design system should be consistent across all NEXUS modules.

System	Guideline
Typography	Clear hierarchy with strong readability for technical content.
Color	Neutral dark/light foundation with semantic status colors reserved for states.
Spacing	Consistent compact spacing for information-dense workstation UI.
Components	Reusable buttons, cards, tabs, drawers, dialogs, tables, command bars.
Icons	Consistent icon language with text labels for critical actions.
Motion	Subtle transitions; never distract from task output.
Code	Monospace code blocks with copy and diff affordances.
Accessibility	Keyboard navigation, focus states, readable contrast and scalable text.

13. Responsive / Window Behavior

NEXUS is primarily a desktop workstation. Layouts should gracefully adapt to smaller windows without pretending to be a mobile-first application.

Window Size	Behavior
Large	Three-column workspace where contextual panel is available.
Medium	Collapse right panel into drawer; preserve main workspace.
Small	Single primary workspace with navigation drawer and contextual overlays.

14. Global Navigation & Command System

A global command/search interface should allow power users to quickly access projects, conversations, models, knowledge, agents and actions.

Example commands: Open Project • New Chat • Search Knowledge • Run Benchmark • View Active Agents • Switch Model • Open Settings.

Keyboard Strategy

Common navigation and action shortcuts should be documented and customizable later. Keyboard accessibility is especially important for coding-agent workflows.

15. Notification & Status System

Status	UI Treatment
Local/Ready	Persistent runtime indicator.
Model Loading	Progress/status without blocking unrelated UI.
Agent Running	Task indicator and activity notification.
Approval Required	High-priority actionable notification.
Ingestion Complete	Knowledge job notification.
Error	Actionable error with recovery path.
Offline	Clear offline indicator; do not imply cloud availability.

16. Empty, Error & Recovery UX

Empty states should teach the user what to do next. Example: no models → connect/check Ollama. No projects → create project. Empty knowledge collection → import documents. No agents → start a task.

Errors must contain: what happened, impact, suggested action and optional technical details. Avoid generic “Something went wrong” messages.

17. Security UX

Security should be visible but not intimidating. Use clear permission explanations for filesystem, terminal, network and integrations.

Example: “NEXUS wants to run `npm test` inside Project X. This command can read/write files in the project workspace. Approve once / Always allow for this project / Reject.”

Never present a model-generated explanation as proof that an action is safe; the permission engine remains authoritative.

18. User Journey — First Launch

Install → Launch → Runtime Check → Ollama Detected → Models Discovered → Quick Introduction → Create/Select Project → Start Chat → Optional Knowledge Import → Ready.

First-Run Success

The user should reach a successful local AI response quickly, then discover advanced capabilities progressively.

19. User Journey — Coding Agent

Project → Chat → User asks for code change → NEXUS proposes/starts task → Agent Workspace opens → Plan → Files inspected → Approval if needed → Edits → Tests → Verification → Diff + Summary.

The user should always be able to see what changed and whether tests actually ran.

20. User Journey — RAG

Knowledge Center → Create Collection → Import documents → Ingestion progress → Indexed → Chat asks question → RAG retrieves evidence → Answer → Source cards → Open source document.

A citation should be actionable: clicking it opens the relevant document/section when available.

21. Application State Model

State	Examples
Runtime	Ollama available, DB healthy, offline/online policy.
Session	Current user/session and permissions.
Project	Current project and workspace scope.
Chat	Conversation, messages, generation state.
Knowledge	Collection, ingestion and search state.
Model	Registry, loading and routing state.
Agent	Task, step, tool and verification state.
UI	Navigation, panels, filters and dialogs.

The frontend must avoid duplicating authoritative backend state. Streaming/event updates should reconcile into the frontend state model.

22. Frontend-to-Backend Interaction

UI Action → Typed API Request → Backend Service → Event/Response → State Update → UI.

For long-running tasks, the frontend subscribes to task events rather than polling aggressively. The event stream updates agent progress, model generation, retrieval and tool status.

23. UI Data Contracts

Major UI objects include Project, Conversation, Message, Model, KnowledgeCollection, Document, IngestionJob, AgentTask, AgentStep, ToolCall, PermissionRequest and SourceReference.

Each object should have stable IDs, status fields and timestamps so screens can safely update from asynchronous events.

24. Performance Requirements

- Initial shell should become interactive quickly.
- Chat responses stream rather than wait for completion.
- Large document lists use pagination/virtualization where necessary.
- Agent activity uses incremental events.
- Code/file viewers avoid loading huge files into the browser at once.
- Knowledge search returns bounded result sets.
- UI remains responsive while local models are loading or agents are running.

25. Accessibility & Usability Requirements

- All primary actions keyboard accessible.
- Visible focus states.
- Semantic labels for controls.
- Readable status messages.
- Color is not the only indicator of state.
- Dialogs clearly identify destructive actions.
- Code and evidence remain selectable/copyable.
- Long-running tasks provide understandable progress.

26. Application Architecture Boundary

The UI is responsible for presentation, interaction and local UI state. The backend/orchestrator remains authoritative for models, tools, permissions, RAG, memory, tasks and execution.

UI must never directly call Ollama for privileged operations or directly execute filesystem/terminal actions. All meaningful operations pass through the application contracts defined by the system architecture.

27. MVP Screen Set

Screen	MVP Priority
Home / Dashboard	P0
Chat	P0
Project Workspace	P0
Knowledge Center	P0
Model Center	P0
Agent Workspace	P1
Settings / Privacy	P0
Benchmark	P1
Advanced Runtime/Diagnostics	P1

28. MVP Application Acceptance Gate

- User can launch the installed application and see local runtime status.

- User can discover and select local Ollama models.
- User can start a conversation and receive streaming output.
- User can create a project and keep project context isolated.
- User can import local documents and see ingestion status.
- User can ask a knowledge question and view source references.
- User can start an agent task and observe its status.
- High-risk tool actions produce explicit approval UI.
- User can inspect privacy/storage/permission settings.
- Errors and offline states are understandable and recoverable.

29. Recommended Frontend Build Order

1. Application shell → 2. navigation/routing → 3. design system → 4. runtime status → 5. chat → 6. project workspace → 7. model center → 8. knowledge center → 9. agent workspace → 10. settings/privacy → 11. command/search → 12. diagnostics and polish.

30. Final Application / UI Statement

NEXUS AI should present advanced local AI as one coherent workstation. Chat is the universal entry point, Projects provide continuity, Knowledge provides evidence, Models provide intelligence choices, Agents provide controlled action, and Settings provide privacy and permission control. The interface must make this complexity discoverable without making it mandatory.

31. Next Engineering Dependencies

N46–N54 convert the system, AI, RAG and agent architectures into the user-facing application. The next implementation documents should define concrete frontend components, desktop packaging/runtime decisions, API contracts, database-to-UI mappings and visual specifications.

Recommended next block: N55–N63 — Application Implementation & Interaction Engineering.

Document: N46–N54 Application / UI Foundation • **Version:** 0.1 • **Status:** Ready for detailed frontend implementation